

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
library upcomp; use upcomp.SubComp.all;
entity Muxnbit2tol is
    generic (width:integer:=8);
    Port ( IO,I1 : in STD_LOGIC_VECTOR
           (width-1 downto 0);
          q : out STD_LOGIC_VECTOR (width-1
                                     downto 0);
          sel : in STD_LOGIC);
end Muxnbit2tol;
```

```
architecture Mnb22l of Muxnbit2tol is
    signal nsel:STD_LOGIC;
begin
    nsel<=not sel;
    TB1:MuBuff generic map(width) port
    map(IO,q,nsel);
    TB2:MuBuff generic map(width) port
    map(I1,q,sel);
end Mnb22l;
```

All the components in the project are added to upcomp library; SubComp is a package in the library that has declaration of all the components for hierarchical designs. The complete SubComp.vhd code is included in EFY DVD of this month. The code is simulated, synthesised, implemented and validated on the Nexys 4 board.

Barrel shifter

Barrel shifter is a shift register that can be designed using multiplexers. Fig. 8 shows a barrel shifter. Bits shift to left or right by the count fed as shift word. LSB bits are fed with 0's when rot bit is low and wrapped around from MSB when rot bit is high. The behavioral description of the barrel shifter is implemented in BarrelShifter.vhd.

ALU

Arithmetic and logic unit is the heart of data path. It performs one of the multiple arithmetic and logic operations selected by the select lines. Table I shows various operations that the ALU performs on two operands 'a' and 'b'. Subscript 'i' represents the i-th bit. Inputs 'a' and 'b' are transformed into 'x' and 'y'

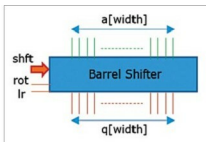


Fig. 8: Barrel shifter

ALU OPERATIONS						
	S ₂	S ₁	S ₀	x _i	y _i	
Pass	0	0	0	a _i	0	
AND	0	0	1	a _i and b _i	0	
OR	0	1	0	a _i or b _i	0	
Complement	0	1	1	not a _i	0	
Add	1	0	0	a _i	b _i	
Subtract	1	0	1	a _i	not b _i	
Increment	1	1	0	a _i	0	
Decrement	1	1	1	a _i	1	

as per the table. Intermediate bits 'x' and 'y' are fed into multibit full adder. Full adder is designed using half adders.

The Transform1bit.vhd code explains bit transformation circuit as per the truth table in the table. The ALU is designed using full adder and bit transform circuit. The code implementation is given in ALUNbit.vhd.

For opcode as 101, the ALU also acts as a comparator. If equ = 1 and agb = 1, 'a' and 'b' are equal. If equ = 0 and agb = 1, 'a' is greater than 'b'. If equ = 0 and agb = 0, 'a' is less than 'b'. The logic also holds for signed numbers.

Fast multiplier

Digital signal processing (DSP) requires fast multiplication of two numbers. If both the numbers are of certain width, the result of multiplication is twice the width. This requires two words that represent lower and higher significance. The output can be multiplexed.

The Multiplier.vhd code below

shows VHDL code for the multibit multiplier:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
library upcomp; use upcomp.SubComp.all;
entity Multiplier is
    generic (width:integer:=8);
    Port ( a,b : in STD_LOGIC_VECTOR
           (width-1 downto 0);
          OP : out STD_LOGIC_VECTOR (width-1
                                     downto 0);
          S0 : in STD_LOGIC);
end Multiplier;
```

Universal shifter

The output of the multiplier or ALU is forwarded through a multiplexer to universal shift register. Universal shift register is capable of bit-by-bit left and right shift with or without rotate option. The basic unit of a register is a D-type flip-flop with asynchronous clear. The code below shows a D-type flip-flop with asynchronous clear:

```
-- D flip flop with clear-DFlipflop.vhd
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity Dflipflop is
    Port ( d,clk,clr : in STD_LOGIC;
          q :inout STD_LOGIC='0');
end Dflipflop;
```

```
architecture DFF of DFlipflop is
begin
    q<='0' when clr='1' else
    d when (clk'event and clk='0' and
           clr='0') else
    q;
end DFF;
```

The shift register also needs 2:1 and 4:1 multiplexers. The code below shows a 2:1 multiplexer:

```
-- 2:1 MUX-MUX2tol.vhd
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity MUX2tol is
    Port (IO,I1,S0 : in STD_LOGIC;
          q : out STD_LOGIC);
end MUX2tol;
```